

Embeddings in LLM Architecture

How a discrete token ID becomes a learned vector — and why retrieval systems train a different kind of embedding entirely.



What you'll be able to do by the end

FOUNDATIONS

- Explain why token IDs alone can't carry meaning
- Define an embedding and an embedding space
- Explain the embedding matrix and embedding lookup
- Calculate embedding parameter counts

HOW THEY'RE LEARNED

- Explain self-supervised learning and next-token prediction
- Explain cross-entropy loss mathematically
- Explain how backpropagation updates embeddings

MEANING & CONTEXT

- Explain why semantically related words develop related vectors
- Distinguish context-independent lookup from contextual representation

BEYOND TOKENS

- Explain sentence/document embeddings and contrastive learning
- Explain positive pairs, negative pairs, and hard negatives
- Explain InfoNCE and triplet loss conceptually
- Distinguish LLM token embeddings from RAG embeddings

Can we feed token IDs straight into a network?

Token	ID
"I"	40
" love"	982
" NLP"	7342

Does 982 mean "love" matters more than "I" because $982 > 40$? Does $982 - 40 = 942$ mean anything at all?

OBVIOUSLY NOT

Token IDs are identifiers, not numerical measurements of meaning.

Token IDs are categorical, not continuous

cat	→ 10
dog	→ 11
car	→ 12
banana	→ 13

We cannot assume $\text{dog} - \text{cat} = 1$ means anything — the network needs a continuous numerical space instead.

cat →	[0.21, -0.17, 0.83, 0.42, ...]
dog →	[0.19, -0.14, 0.79, 0.45, ...]

Close, related values can develop useful relationships during training.

Definition

An embedding is a dense numerical vector representation of an object, learned so that useful relationships between objects can be represented in the resulting vector space.

Token: "cat"



[0.21, -0.18, 0.73, 0.11, ... , 0.04]

The vector may have hundreds or thousands of dimensions.

The embedding matrix

$$\mathbf{E} \in \mathbb{R}^{(V \times d)}$$

where V = vocabulary size, d = embedding dimension

EXAMPLE INPUTS

Vocabulary size $V = 50,000$ tokens
Embedding dimension $d = 768$

PARAMETERS

38,400,000

50,000 × 768

One row per token

	dim 0	dim 1	dim 2	...	dim d
Token 0	.21	-.42	.17	...	.63
Token 1	.10	.18	.72	...	.11
Token 2	-.12	.71	.31	...	.42
...	...	...	...	...	...
Token V	.12	.44	.56	...	.31

Each row is one token's embedding. Each column is one learned dimension of meaning — with no single dimension having a hand-assigned label.

Embedding lookup

Token ID = 582



Embedding matrix E



Row 582

[0.21, -0.18, 0.73, ... , 0.04]

IMPORTANT

This is not a similarity search — it's a direct index: $E[582]$.

We don't hand-pick the vector for “cat”

The model learns it during training. Embeddings start out with no useful semantic structure at all.

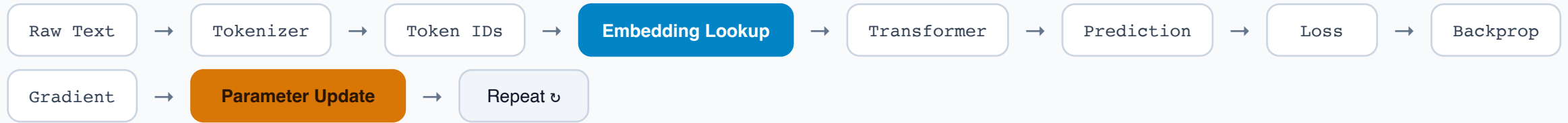
cat → [random-ish vector]

dog → [random-ish vector]

car → [random-ish vector]

Then training begins — and pushes every one of these vectors toward usefulness.

The complete LLM training loop



The embedding matrix is one of the parameters being updated on every pass.

Do we need a hand-labeled dataset?

NOT NEEDED

cat → animal
dog → animal
car → vehicle

WHAT ACTUALLY WORKS

"The cat sat on the mat."
"The dog chased the ball."
"The cat was sleeping near the window."

A language model learns from raw text — this is called self-supervised learning: the data generates its own training target.

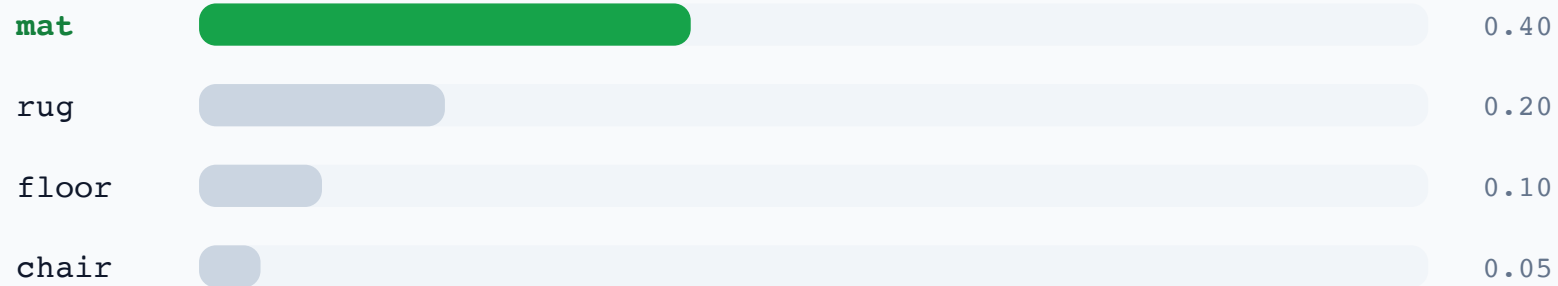
One sentence, four training examples

Input	Target
The	cat
The cat	sat
The cat sat	on
The cat sat on the	mat

"The cat sat on the mat" supplies both the input and the target — no human labels a single example.

Next-token prediction

Input: "The cat sat on the"



The correct answer is "mat" — training pushes its probability higher.

Cross-entropy loss

$$L = -\log P(y)$$

y = correct token · $P(y)$ = probability assigned to it

CONFIDENT & RIGHT · $P(\text{mat}) = 0.8$

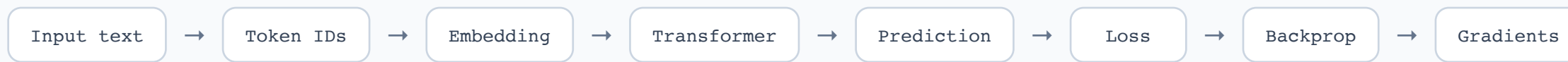
$$L \approx 0.223$$

UNSURE · $P(\text{mat}) = 0.01$

$$L \approx 4.605$$

Training tries to raise the probability of the correct next token, thereby reducing the loss.

From loss back to the embedding



Embedding updated

E_{eat} before

[0.10, 0.40, -0.20, 0.70]

E_{eat} after

[0.11, 0.39, -0.18, 0.72]

Values shift because changing them can help reduce future prediction error.

Gradient descent

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \cdot (\partial L / \partial \theta)$$

θ = model parameters · L = loss · η = learning rate

$$E_{\text{new}} = E_{\text{old}} - \eta \cdot (\partial L / \partial E)$$

The embedding matrix E is itself a parameter.

Why do embeddings develop meaning?

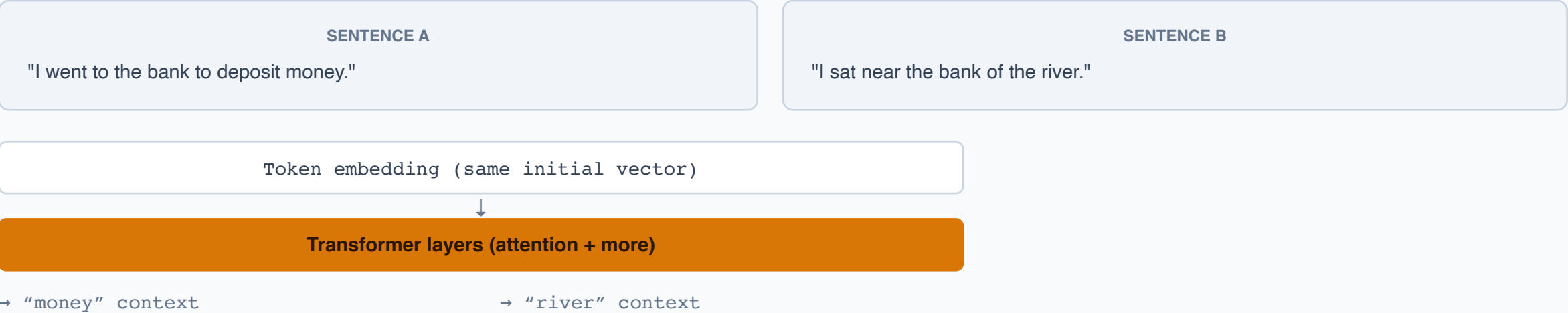
- The ____ barked loudly.
- The ____ chased the ball.
- The ____ ate the food.
- The ____ was sleeping.

Both dog and cat fit these contexts. Across huge quantities of text, the model learns statistical relationships between tokens and their contexts — so related words can develop related vectors.

DON'T SAY “DIMENSION 1 MEANS ANIMAL” OR “DIMENSION 2 MEANS GENDER”

Meaning is distributed across many dimensions, not localized to one.

“bank” is one token, two meanings



Representation	Nature
Initial token embedding	context-independent
Transformer hidden representation	context-dependent

From token embeddings to sentence embeddings

TOKEN EMBEDDING

"cat" → vector

SENTENCE / TEXT EMBEDDING

"I want to return my mixer." → one vector

"I want to return my mixer."



Embedding model

[0.12, -0.83, 0.42, ... , 0.17]

Now this vector can be compared against document vectors — the foundation of RAG retrieval.

Contrastive learning: pull relevant close, push irrelevant apart

Query: "How can I return my mixer?"

positive

"Products can be returned within 30 days."

negative

"Orders are delivered within 3–5 days."

Similarity(Query, Return Policy)

HIGH

Similarity(Query, Shipping Policy)

LOW

Positive, negative, and hard-negative pairs

positive

Return policy — directly relevant

negative

Shipping policy — clearly irrelevant

hard negative

Mixer warranty policy — looks related, is wrong

Hard negatives force the model to learn finer semantic distinctions, not just topic-level similarity.

Contrastive loss, conceptually

POSITIVE PAIR

Bring vectors closer

NEGATIVE PAIR

Push vectors apart

$$L = -\log \left[\frac{e^{\text{sim}(q, d^+)/\tau}}{\sum_j e^{\text{sim}(q, d_j)/\tau}} \right]$$

q = query · d^+ = positive doc · d_j = candidate docs · τ = temperature

This is InfoNCE-style loss. Focus on the intuition, not the derivation.

Anchor, positive, negative

Role	Example
Anchor	"I want to return my mixer."
Positive	"Products can be returned within 30 days."
Negative	"Orders arrive within 3–5 days."

```
sim(anchor, positive) > sim(anchor, negative)
```

```
L = max( 0, d(a,p) - d(a,n) + m )
```

m = margin

LLM token embedding vs. RAG embedding

	LLM token embedding	Text / RAG embedding
Input	Token	Sentence / query / document
Purpose	Feed the Transformer	Semantic retrieval
Training objective	Next-token prediction	Contrastive / ranking
Typical loss	Cross-entropy	InfoNCE / triplet
Output	Token representation	Text vector
Used for retrieval?	No	Yes
Vector DB?	Not normally	Common
Positive / negative pairs	Not required	Common

Daily Essentials: “Can I send my mixer back?”

1

Query embedding — "I bought a mixer 20 days ago. Can I send it back?" → embedding model → query vector.

2

Documents already embedded — Return, Shipping, Warranty, Refund, and Payment policies were each embedded at indexing time.

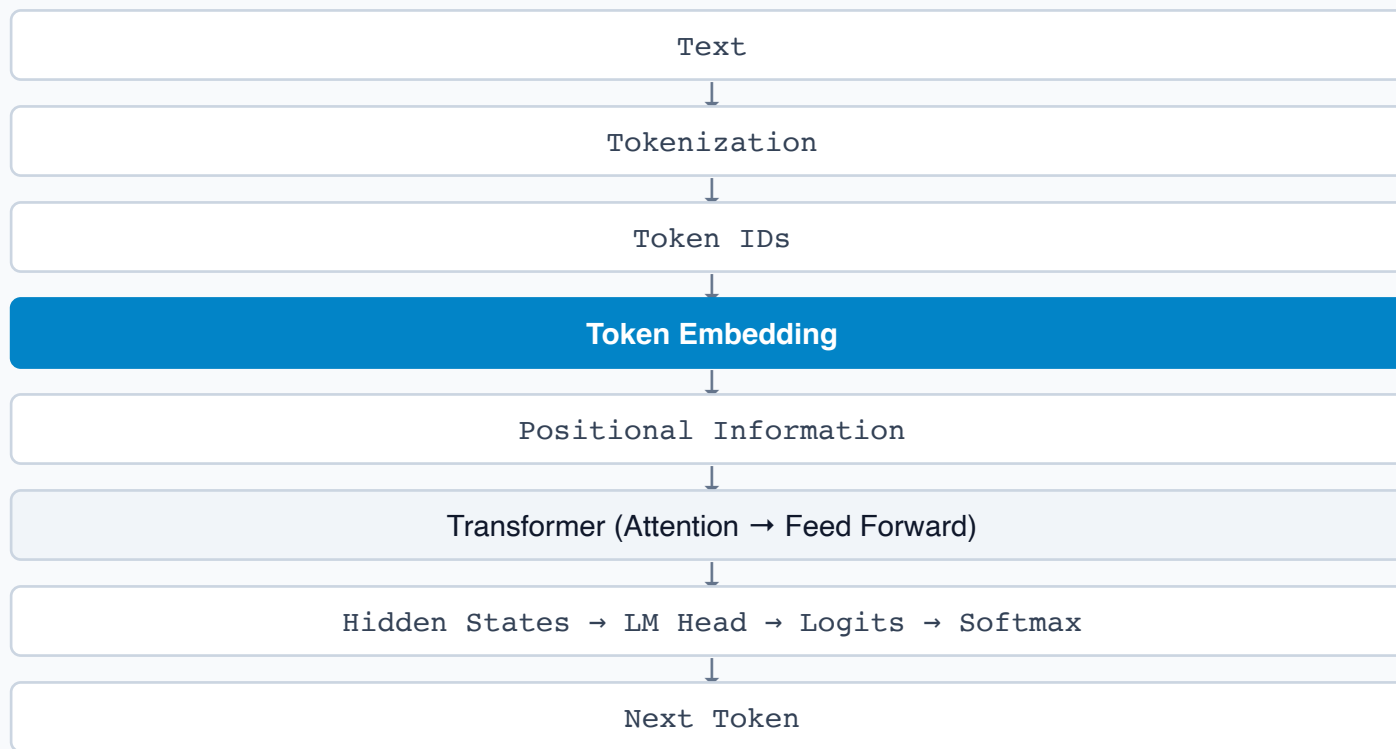
3

Similarity search — Query vector vs. vector database → top-k documents: 1. Return Policy · 2. Refund Policy · 3. Warranty Policy.

4

RAG — Retrieved documents + query are passed to the LLM, which generates the final answer.

Embeddings inside the full LLM architecture



Today we understand the bottom part. Next: how these token representations interact with each other — Attention.

Embedding matrix: $V = 30,000, d = 512$

1. Matrix size?

30,000 × 512

2. Number of parameters?

15,360,000

3. Sequence [42, 100, 928, 7312] — how many vectors?

4

4. Is token 7312 "more meaningful" than token 42?

No

Which prediction gets more loss?

$P(\text{mat}) = 0.8$

$L \approx 0.223$

$P(\text{mat}) = 0.1$

$L \approx 2.303$

The $P(\text{mat}) = 0.1$ case receives greater loss — the model was less confident in the correct answer.

Two vectors, two very different jobs

LLM TRAINING LOOP

Text → Tokenize → Embed → Transformer
→ Logits → Softmax → Next token
→ Loss → Backprop → Update (incl. embeddings)

RAG RETRIEVAL LOOP

Document → Embedding model → Document vector
→ Vector DB
Query → Embedding model → Query vector → Vector DB

What to leave with

1

Tokenization converts text into discrete IDs; embeddings convert those IDs into learned continuous vectors.

2

LLM token embeddings are learned indirectly through the language-model objective — typically next-token prediction with cross-entropy loss.

3

Dedicated retrieval embeddings are often trained with a different objective: make relevant texts close together and irrelevant texts far apart.

Ten questions to close

Q1. What is a token ID?

An arbitrary categorical identifier

Q2. Why can't we treat token IDs as meaning?

ID distance has no semantic interpretation

Q3. What is an embedding?

A learned dense vector representation

Q4. What is the embedding matrix?

$E \in \mathbb{R}^{(V \times d)}$

Q5. How does an LLM learn token embeddings?

Via the training objective and backpropagation

Ten questions to close, continued

Q6. Typical objective for an autoregressive LLM?

Next-token prediction

Q7. What is the typical loss?

Cross-entropy / negative log-likelihood

Q8. Are initial token embeddings contextual?

No — the lookup is context-independent

Q9. Where does contextual meaning arise?

Transformer processing, incl. attention

Q10. How are RAG embedding models often trained?

Contrastive / ranking objectives with pos. & neg. examples

We now have a vector for every token.

But a vector for "bank" by itself doesn't say whether we mean a financial bank or a river bank. How does the model look at the other tokens and decide which information is relevant?

Lecture 7 —
Embeddings



**Lecture 8 — Attention
Mechanism**